14 OpenClaw AI Agents Running 100% Locally

on a Single RTX 3090 GPU

52ms, Not 6 Minutes

Zero cloud APIs. Zero subscriptions. Full reasoning on one GPU from 2020.

6,667x faster with KV cache offloading

Andrew Darius
AIgenteur Academy
skool.com/aigenteur-academy-9764

A home computer from 2019 with an RTX 3090 GPU from 2020 - running **Qwen3.5** models released this year. No cloud API. No subscription. Everything runs on one machine.

	Without Optimization	With KV Cache + Agent Routing
200K context prompt	5.8 minutes (9B)	52 ms (9B)
200K context prompt	2.4 minutes (2B)	21 ms (2B)
Speedup		6,667× faster

> *Same GPU. Same model. Just smarter caching. The KV cache stores the system prompt so it's only processed once - every subsequent request skips straight to the new tokens.*

The Problem

We have **14 AI agents** with very different needs:

- Some need **deep reasoning** (strategy, compliance, code architecture)
- Some need **raw speed** (trend scouting, short copy, voice workflows)

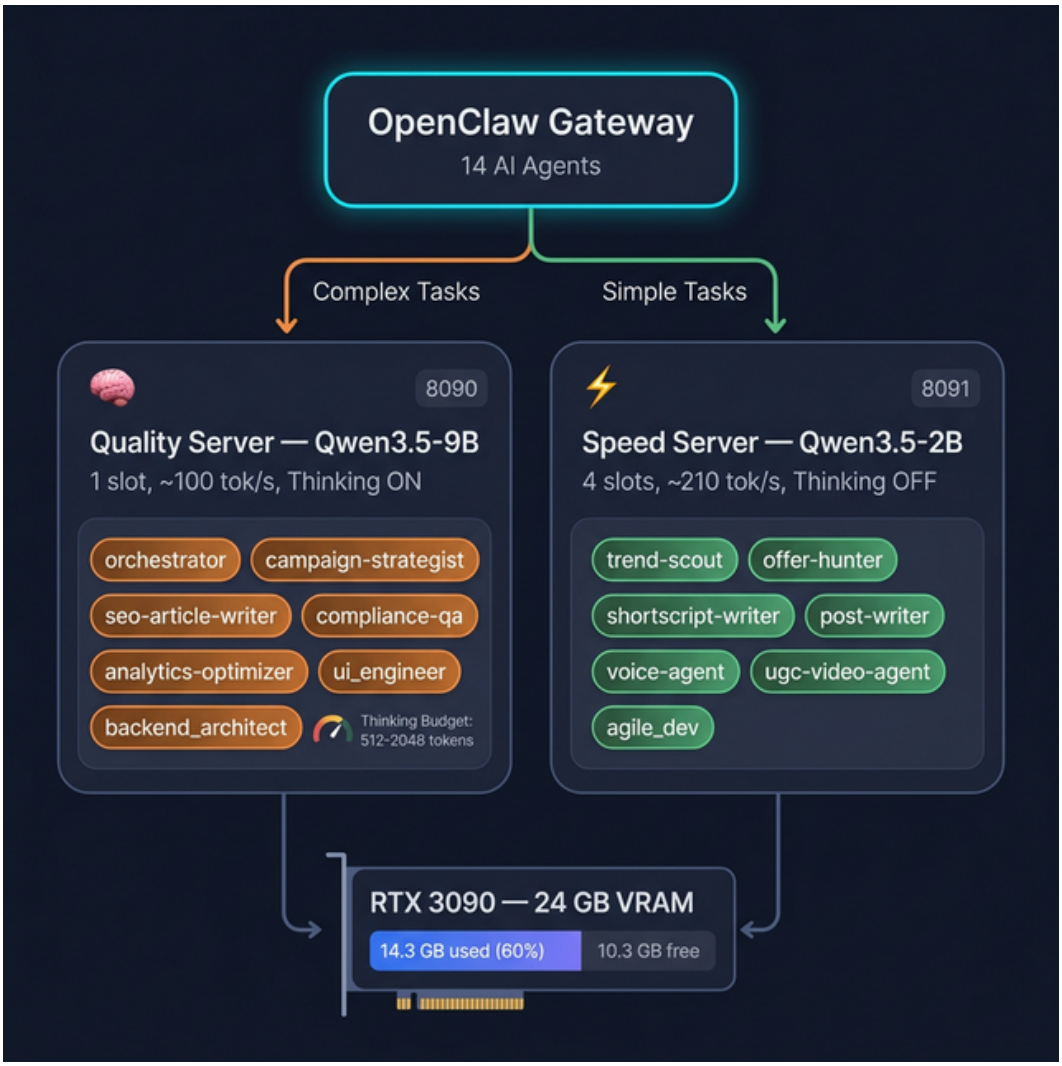
Running all 14 on a single large model wastes GPU on simple tasks. Running them all on a small model degrades quality on complex ones.

The Solution: Two Models, One GPU

We run **two LLM servers simultaneously** on a single RTX 3090 (24 GB), each optimized for its role:

	Quality Server	Speed Server
Model	Qwen3.5-9B	Qwen3.5-2B
Port	8090	8091
Thinking	ON (budgeted)	OFF
Slots	1 (serial)	4 (parallel)
Context	262K tokens	262K tokens
Speed	~100 tok/s	~210 tok/s
VRAM	~10.4 GB	~4.0 GB
Total	14.3 GB used	10.3 GB free

Architecture Overview



OpenClaw routes complex tasks to the 9B server and simple tasks to the 2B server

What is "Thinking" and Why Budget It?

Qwen3.5 models can **reason internally** before answering. The model generates hidden "thinking tokens" inside `<think>...</think>` tags, then produces the actual answer.

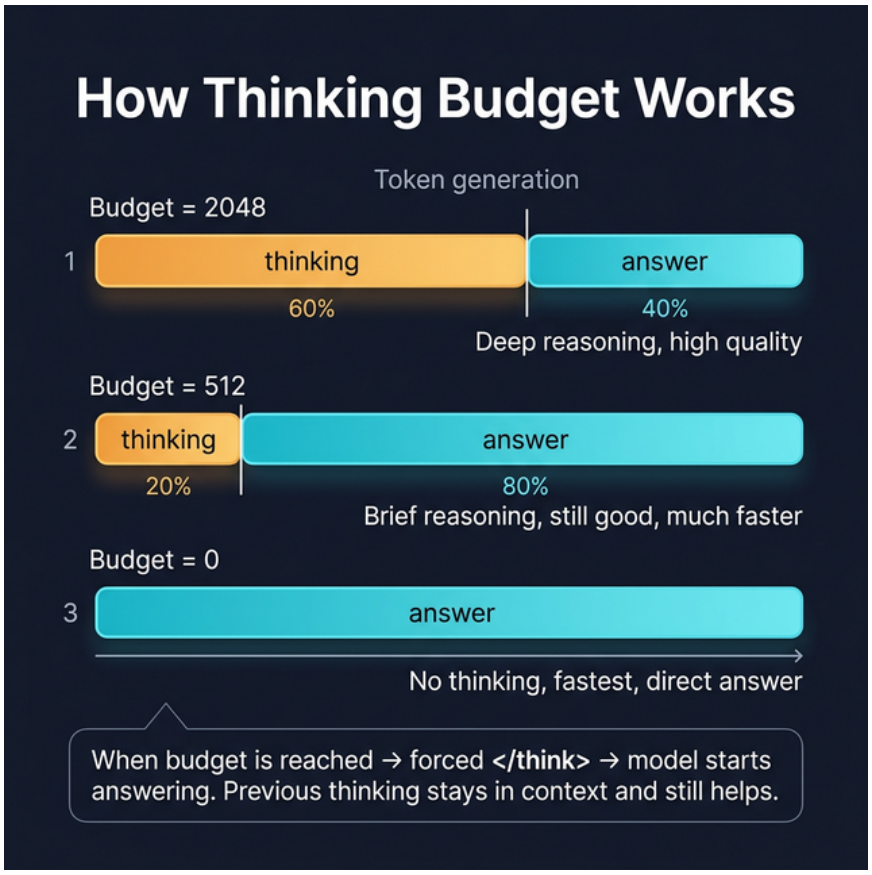
Example:

```
User: "Review this ad for compliance issues" Model thinking (hidden): 1. Check claims...
"50% off" needs substantiation 2. Check disclaimers... missing terms link Model answer
(visible): "Two issues found: 1) The '50% off' claim needs..."
```

The tradeoff: More thinking = better answers, but slower and uses more of max_tokens.

Budget	Behavior	Use Case
0	No thinking at all	Fast tasks: routing, short copy
512	Brief reasoning	Content writing, SEO
1024	Moderate reasoning	Compliance, analytics, UI code
2048	Deep reasoning	Orchestration, backend architecture

*IMPORTANT: **max_tokens** includes BOTH thinking AND answer tokens. Formula: **max_tokens = thinking_budget + expected_output***



Agent Assignment

[Quality] Server (9B, port 8090) — 7 agents

Tasks where **quality matters more than speed**. The 9B model thinks before answering.

#	Agent	Think Budget	Output	max_tokens	Role
1	main (orchestrator)	2048	2048	4096	Routes & coordinates
4	campaign-strategist	1024	3072	4096	Strategy & positioning
7	seo-article-writer	512	6144	8192	Long-form SEO articles
10	compliance-qa	1024	2048	3072	Policy & compliance
11	analytics-optimizer	512	2048	3072	Performance analysis
12	ui_engineer	1024	8192	9216	Frontend/UI code
13	backend_architect	2048	8192	10240	Backend & API code

** Code agents get larger budgets: code generation needs more thinking to plan the approach and larger output because a single component can be 200-400 lines.*

■ Speed Server (2B, port 8091) — 7 agents

Tasks where **speed matters more than depth**. No thinking overhead.

#	Agent	Thinking	max_tokens	Role
2	trend-scout	OFF	2048	Trends & market signals
3	offer-hunter	OFF	2048	Competitive deals
5	shortscript-writer	OFF	2048	Short-form scripts
6	post-writer	OFF	2048	Social posts & copy
8	voice-agent	OFF	1024	Voice workflows
9	ugc-video-agent	OFF	2048	UGC & video concepts
14	agile_dev	OFF	4096	Bug fixes, small patches

** agile_dev stays on 2B: it handles small, scoped patches. For full features, backend_architect on 9B takes over. 2B is fast (210 t/s) keeping the dev loop tight.*

Per-Request Thinking Control

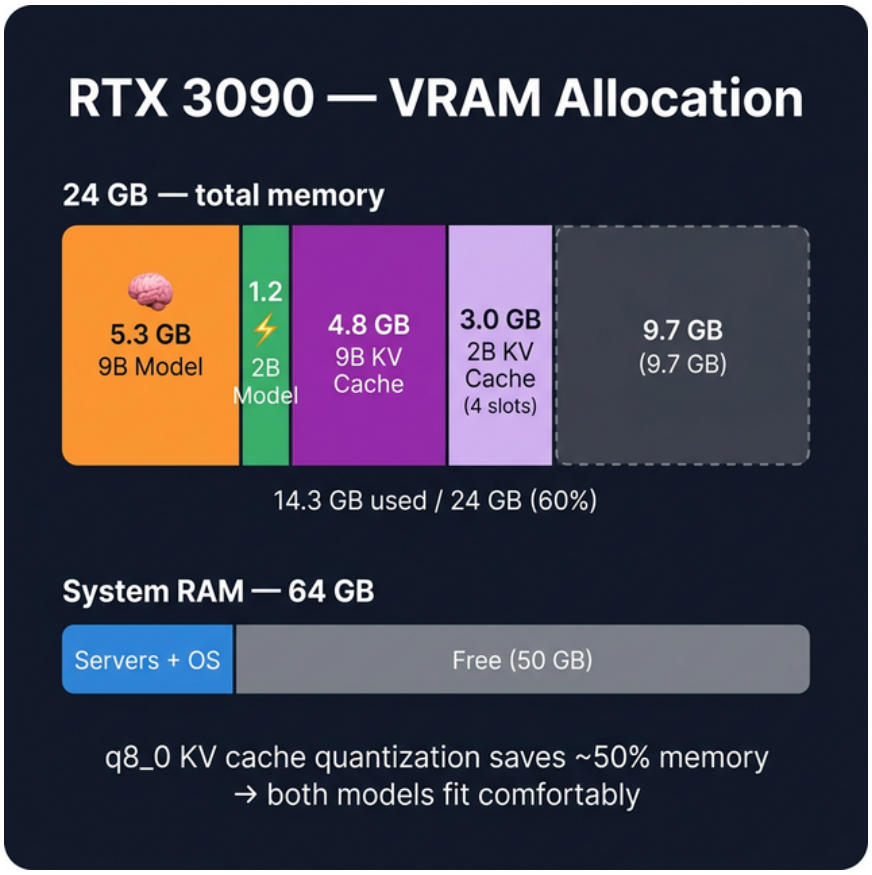
Each agent sets its own thinking budget via the API:

```
# Compliance agent (9B) — needs deep thinking POST
http://server:8090/v1/chat/completions { "reasoning_budget": 1024, ← up to 1024
thinking tokens "max_tokens": 3072 ← 1024 thinking + 2048 answer } # Post writer (2B) —
no thinking needed POST http://server:8091/v1/chat/completions { "reasoning_budget": 0,
← no thinking at all "max_tokens": 512 ← pure answer output }
```

*When the thinking budget is reached, the model is **forced to stop thinking** and immediately start producing the answer. The thinking it already did still helps — it stays in the model's context, so even partial thinking improves answer quality.*

Resource Summary

Resource	Value
GPU	RTX 3090, 24 GB VRAM
VRAM used	14.3 GB (60%)
VRAM free	10.3 GB (headroom for KV cache growth)
System RAM	64 GB (~6 GB OS + ~8 GB KV cache swap = ~14 GB used, 50 GB free)
Context window	262K tokens per server
KV cache	q8_0 (8-bit, saves ~50% VRAM vs default)
Total agents	14
Models loaded	2 (simultaneously)



VRAM breakdown: both models + KV cache fit with 40% headroom

Want the Full Implementation Guide?

This presentation showed WHAT we built. The full guide shows exactly HOW - every patch, every config flag, every optimization explained step by step.

Join the free AIgenteur community to get it:

https://www.skool.com/aigenteur-academy-9764

Inside you will find: patched llama.cpp source, custom Jinja templates, server launch scripts, budget enforcement code, and deployment configs.

AIgenteur Academy

Local AI. Real hardware. No cloud.